

Customer Churn Prediction - Project Report

Author: Punith (Puneeth M)

Repository: github.com/Puneeth7104/Customer-Churn-Prediction

Date: 4 October 2026

1. Problem Statement and Objective

Subscription businesses lose revenue when customers cancel. Acquiring a new customer typically costs more than retaining an existing one, so identifying likely churners early lets the business target retention effort where it matters. The objective of this project is to build a machine-learning system that (a) predicts the probability that each customer will churn, (b) compares several models using suitable metrics, (c) deploys the best model, and (d) produces a ranked list of high-risk customers with suggested review reasons.

2. Data

No real dataset was provided, so a synthetic dataset of 5,000 customers was generated with `src/generate_data.py` (fixed random seed 42 for reproducibility). Because it is synthetic it contains no confidential information and can be shared publicly. The churn label is generated from a hidden logistic process driven by realistic factors (contract type, tenure, usage decline, support contacts, satisfaction, late payments, price increases, discounts), plus randomness, so the problem is learnable but not trivially separable. Overall churn rate is 30.2%.

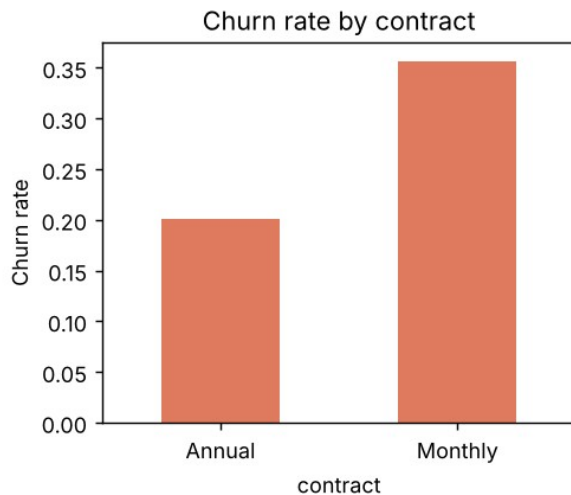
Data groups collected: demographics (age, gender, region), account (plan, contract, payment method, tenure), usage (hours in each of the last 3 months, logins in 30 days), support interactions (tickets in 90 days, average resolution days, satisfaction score), and billing history (monthly charges, previous charges, late payments, discount flag).

Realistic data-quality issues were deliberately injected to demonstrate cleaning: missing values in `satisfaction_score` (6%), `avg_resolution_days` (3%), `age` (2%) and `usage_hours_m2` (2%), plus billing and usage outliers.

3. Exploratory Findings

- Monthly-contract customers churn at 35.7% versus 20.1% for annual contracts.
- Churn falls steadily with tenure: 39.3% for tenure up to 6 months, 32.8% for 7-12 months, 28.6% for 13-24 months and 18.8% beyond 24 months.

- Customers with falling usage, more support tickets, low satisfaction and repeated late payments churn more.



4. Data Preparation

- Missing values: median imputation for numeric columns (fit on training data only, inside the pipeline, to avoid leakage).
- Categorical variables: one-hot encoding (gender, region, plan, contract, payment method); unknown categories are ignored at prediction time.
- Outliers: monthly charges and usage hours capped at fixed upper limits so extreme values do not dominate (important for Logistic Regression).
- Scaling: standardisation for Logistic Regression only; tree models do not need it.
- Class imbalance (about 30% positives): `class_weight='balanced'` for Logistic Regression, `'balanced_subsample'` for Random Forest, and `scale_pos_weight` for XGBoost. A stratified 80/20 train-test split keeps the churn ratio equal in both sets.

5. Feature Engineering

- `usage_trend = (usage last month - usage 3 months ago) / (usage 3 months ago + 1)`: negative values mean declining engagement.
- `usage_avg_3m`: average monthly usage over 3 months.
- `support_freq_per_month = tickets in 90 days / 3`.
- `price_change_pct`: recent change in monthly charges versus the previous bill.
- `charges_per_tenure`: monthly charges relative to how long the customer has stayed.
- `low_engagement`: flag for very low usage hours or logins.
- `payment_risk`: flag for 2 or more late payments in 6 months.

6. Modelling Approach

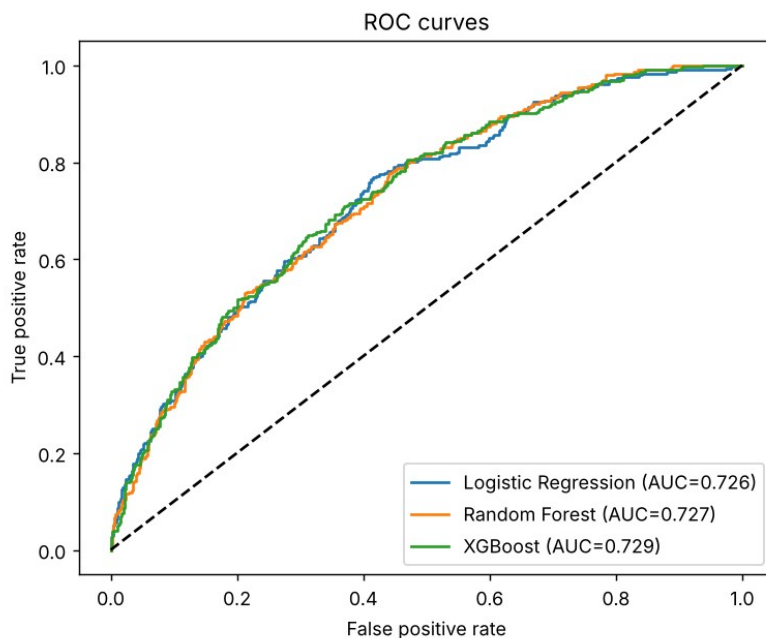
Three models were trained on identical features and the same split: Logistic Regression (interpretable baseline), Random Forest (300 trees, min leaf 5) and XGBoost (300 trees, depth 4, learning rate 0.05, subsample 0.8). All preprocessing sits inside scikit-learn Pipelines so the same transformation is applied at training and prediction time.

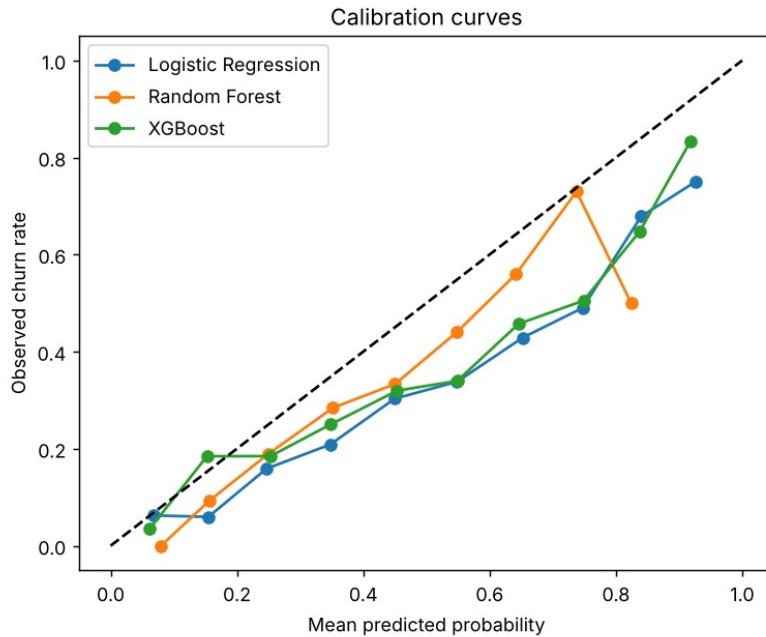
7. Results and Model Comparison

Metrics on the 1,000-customer hold-out set at a 0.5 threshold:

Model	ROC-AUC	Precision	Recall	F1	Brier (calibration)
Logistic Regression	0.726	0.449	0.675	0.540	0.211
Random Forest	0.727	0.524	0.477	0.499	0.189
XGBoost	0.729	0.478	0.613	0.537	0.202

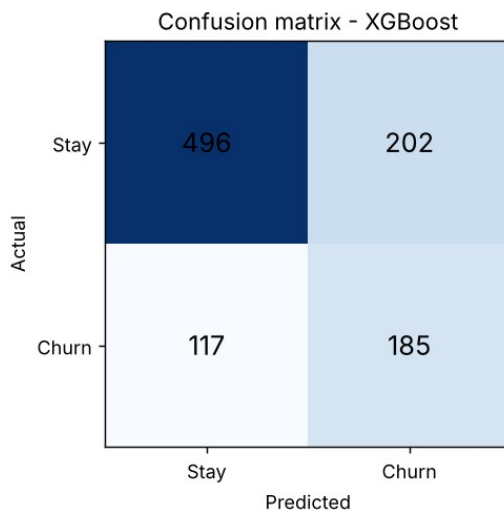
XGBoost obtained the highest ROC-AUC (0.729) and was selected as the final model. The three models are very close in AUC (about 0.73), which is expected because the underlying churn signal is mostly additive and noisy; this also means the simple Logistic Regression baseline is a strong, explainable alternative. Random Forest has the best precision and the lowest Brier score (best calibration), but the lowest recall. Logistic Regression and XGBoost catch more churners (recall 0.68 and 0.61). Which model is 'best' therefore depends on business cost: if missing a churner is costly, favour recall; if retention offers are expensive, favour precision.





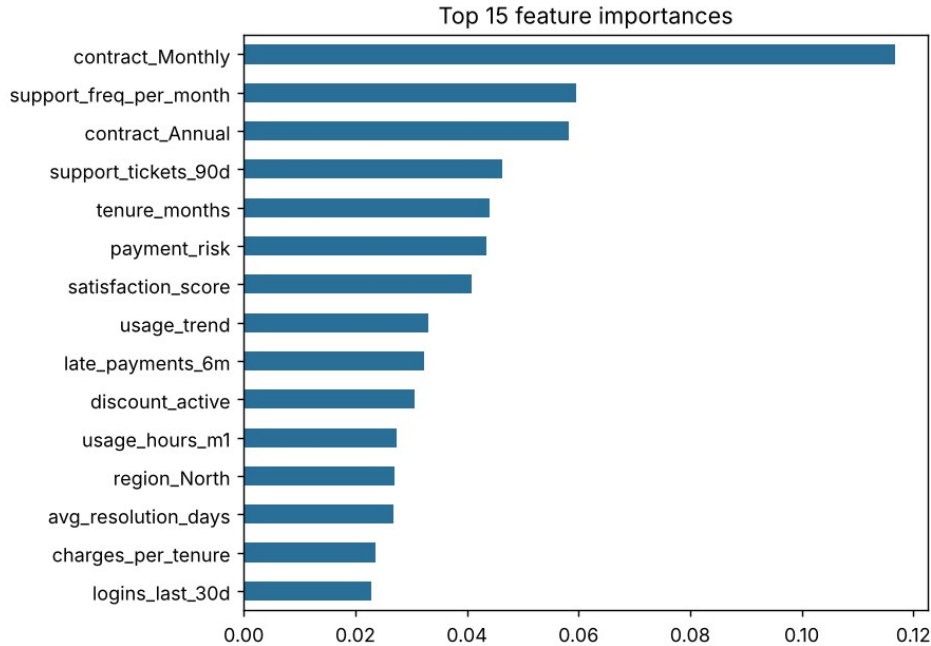
Calibration: Random Forest probabilities are closest to observed frequencies (lowest Brier score). Class weighting pushes the weighted models' probabilities upward, so their raw probabilities should be read as risk scores for ranking rather than exact likelihoods. If exact probabilities are needed, the next step is probability calibration (isotonic or Platt).

Confusion matrix for XGBoost (threshold 0.5): 496 true negatives, 202 false positives, 117 false negatives, 185 true positives.



Business view: ranking the hold-out customers by predicted risk, the top 10% contain 62% actual churners versus a base rate of 30%, a lift of 2.1x. Contacting the highest-risk decile is therefore about twice as efficient as contacting customers at random.

8. Feature Importance



Contract type is the strongest driver, followed by support-contact frequency, tenure, payment risk and satisfaction score. This matches the EDA and gives the business concrete levers: move monthly customers to annual plans, fix support experience, and watch payment behaviour.

9. Deployment and Ranked High-Risk List

The best model is saved as `models/best_model.joblib` and served through a Streamlit app (`app.py`) with three tabs: single-customer prediction with reasons, the ranked list with filters and CSV download, and model results. Every customer was scored and ranked in `outputs/ranked_high_risk_customers.csv` with churn probability, risk tier (High above 0.7, Medium 0.4-0.7, Low below 0.4), review reasons and a suggested retention action. 941 customers are High risk and 1488 Medium.

Review reasons are rule-based and transparent (for example: usage dropped sharply, frequent support contacts, low satisfaction, repeated late payments, recent price increase, month-to-month contract). Suggested actions map those reasons to retention plays such as a loyalty discount, a priority support callback, a re-engagement campaign or an annual-plan incentive.

10. Limitations

- The data is synthetic, so the absolute metrics describe this simulated world, not a real business. The pipeline is designed to run unchanged on a real dataset with the same columns.

- The ranked list scores all customers including those used for training; the honest performance estimate is the hold-out result in Section 7.
- A 0.5 threshold was used. In practice the threshold should be chosen from the cost of a retention offer versus the value of a retained customer.
- Review reasons are rule-based, not model-derived explanations; SHAP values would be a natural upgrade.

11. Future Work

- Replace synthetic data with real customer data and re-run the pipeline.
- Hyperparameter tuning with cross-validation and probability calibration.
- SHAP-based per-customer explanations.
- Scheduled retraining and monitoring of data drift.
- A/B test retention actions to measure real uplift.

12. How to Reproduce

`pip install -r requirements.txt; python src/generate_data.py; python -m src.train; streamlit run app.py`